Experiment No. 2
Design & Implementation of Pass-2 of Two Pass Assembler
Date of Performance:16/03/26
Date of Submission:30/03/26

Aim: Design and Implementation of Pass 2 of Two Pass Assembler.

Objective: To study and implement Pass 2 of two pass assembler for IBM 360 Machine.

Theory:

An assembler is a program that accepts as input an assembly language program (source) and produces its machine language equivalent (object code) along with the information for the loader. The function of pass 2 of assembler is to generate machine program which is semantically equivalent to source program by taking input from pass 1. Databases generated are used to generate object code in pass 2.

Pass2: Generate object program

1. Look up value of symbols [STGET]
2. Generate instruction [MOTGET2]
3. Generate data (for DS, DC & literals)
4. Process pseudo ops [POTGET2]

Pass2: Databases

1. Copy of source program input to Pass1.
2. Location Counter (LC)
3. MOT [Mnemonic, length, binary m/c op code, etc.]
4. POT [Mnemonic & action to be taken in Pass2
5. ST [prepared by Pass1, label & value]
6. Base Table [or register table] indicates which registers are currently specified using 'USING' pseudo op & what are contents.
7. Literal table prepared by Pass1. [Lit name & value].

Flowchart of Pass 2:

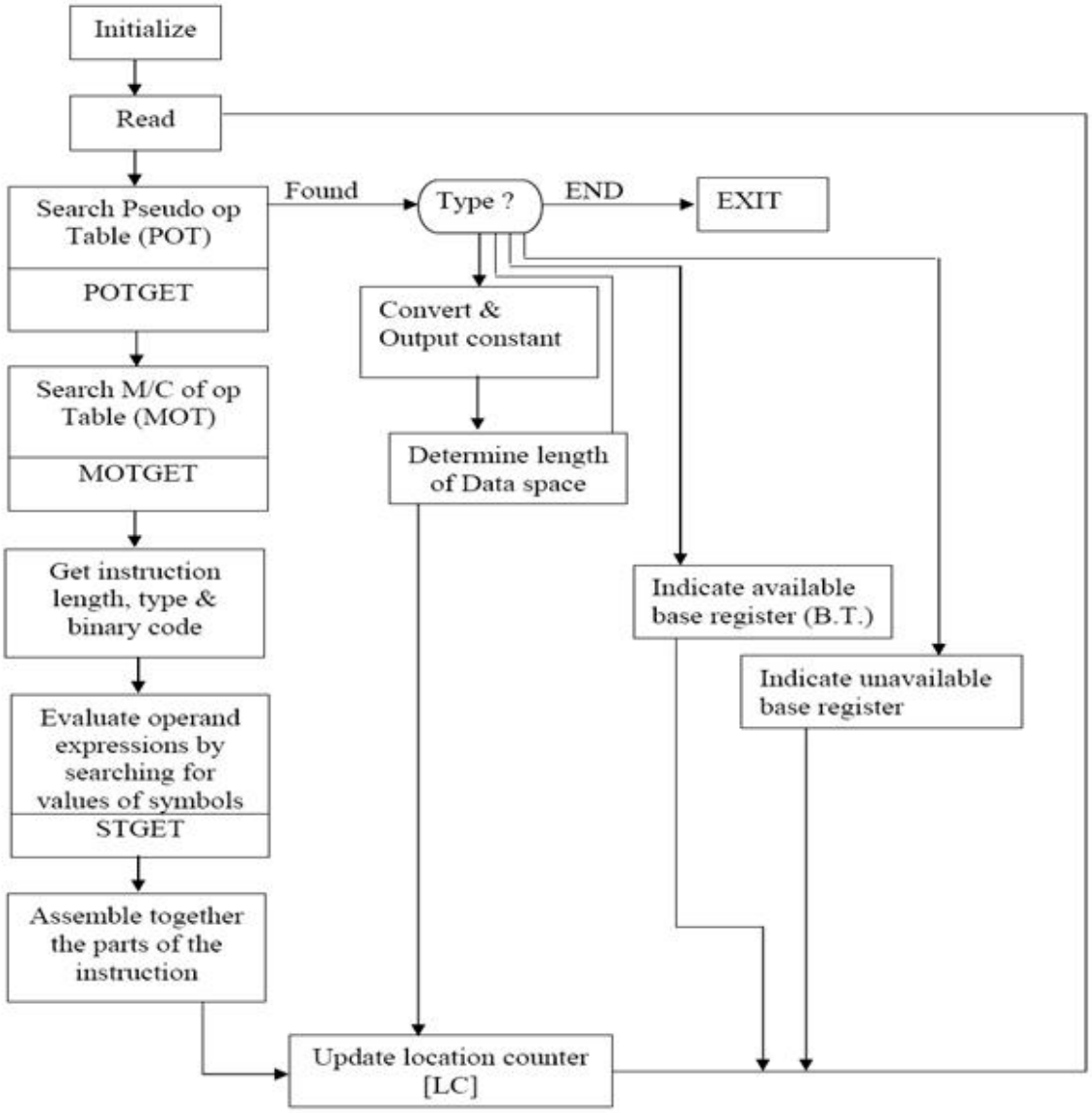

Fig. Flow chart of Pass -2

Code:

```
#include <stdio.h>

#include <string.h>

int main()

{
```

```
int symtab[10], littab[10];
int symCount, litCount;
int i;

int lc = 0;
char type[3];
int opcode;
char opType;
int opValue;

/* -------- Symbol Table Input -------- */
printf("\nEnter number of symbols: ");
scanf("%d",&symCount);

printf("Enter symbol addresses:\n");
for(i=0;i<symCount;i++)
    scanf("%d",&symtab[i]);

/* -------- Literal Table Input -------- */
printf("\nEnter number of literals: ");
scanf("%d",&litCount);

printf("Enter literal addresses:\n");
for(i=0;i<litCount;i++)
    scanf("%d",&littab[i]);

printf("\n---------------------------------\n");
printf("Enter Intermediate Code\n");
printf("Example: (IS,04) (R,1) (S,1)\n");
printf("Enter opcode -1 to stop\n");
```

```
printf("---------------------------------\n");

printf("\nLC   OP  REG  ADDR\n");

while(1)
{
    scanf(" (%[^,],%d)",type,&opcode);

    if(opcode==-1)
        break;

    /* ----- Imperative Statement ----- */
    if(strcmp(type,"IS")==0)
    {
        int reg;
        char addrType;
        int addrIndex;

        scanf(" (R,%d) (%c,%d)",&reg,&addrType,&addrIndex);

        if(addrType=='S')
            printf("%03d %02d  %d   %03d\n",lc,opcode,reg,symtab[addrIndex-1]);

        else if(addrType=='L')
            printf("%03d %02d  %d   %03d\n",lc,opcode,reg,littab[addrIndex-1]);

        lc++;
    }

    /* ----- Declarative Statement ----- */
    else if(strcmp(type,"DL")==0)
    {
```

```
        int constant;
        scanf(" (C,%d)",&constant);

        printf("%03d 00  0   %03d\n",lc,constant);
        lc++;
    }

    /* ----- Assembler Directive ----- */
    else if(strcmp(type,"AD")==0)
    {
        int value;
        scanf(" (C,%d)",&value);

        if(opcode==1)   // START directive
            lc=value;
    }
  }

  printf("\nPass-2 Execution Completed.\n");

  return 0;
}
```

Output:

```
Enter number of symbols: 3
Enter symbol addresses:
200
205
210

Enter number of literals: 2
Enter literal addresses:
300
305

---------------------------------
Enter Intermediate Code
Example: (IS,04) (R,1) (S,1)
Enter opcode -1 to stop
---------------------------------

LC   OP  REG  ADDR
(IS,04) (R,1) (S,1)
(IS,01) (R,2) (S,2)
(IS,02) (R,3) (L,1)
(DL,01) (C,5)
(AD,01) (C,100)
(IS,-1)
000 04  1   200
001 01  2   205
002 02  3   300
003 00  0   005

Pass-2 Execution Completed.
```

Conclusion:

1. Comment on working of Program by providing input source program.
2. Comment on databases generated during Pass 1 of Two Pass assembler.

Comment on working of Program by providing input source program.

The working of a Two Pass Assembler program can be understood by providing an input source program written in assembly language. In the first pass, the assembler reads the source program line by line and assigns addresses to each instruction or statement using the location counter (LC). It also identifies labels and stores them in the symbol table. During this process, it checks the syntax of the program and records necessary information for further processing. In the second pass, the assembler uses the information generated in Pass 1 to translate the assembly instructions into machine code. Finally, it generates the object code and listing file which can be executed by the machine.

Comment on databases generated during Pass 1 of Two Pass assembler.

During Pass 1 of a Two Pass Assembler, several important databases are created to assist in the translation process. The most important database is the Symbol Table (SYMTAB), which stores the symbols or labels along with their assigned addresses. Another database is the Literal Table (LITTAB), which keeps track of all literals used in the program and their addresses. The Pool Table (POOLTAB) is also generated to maintain the starting index of each literal pool. Additionally, the assembler maintains an intermediate code file that stores the processed form of the source program. These databases help the assembler efficiently generate correct machine code during Pass 2.